

WORKSHOP 2 · HANDS-ON TRAINING

Applied Python Programming and Core AI Libraries

Building fluency in core Python constructs, then applying them with pandas, matplotlib, scikit-learn, TensorFlow/Keras and PyTorch.

Undergraduate Engineering & Applied AI | Follow-up to Workshop 1: Environment & Tooling

From environment to execution

You already have

- VS Code configured as your Python IDE
- uv for fast, reproducible environments
- Git for version control and collaboration
- WSL2 as a Linux-native runtime
- Docker for portable, shareable setups
- Python version pinned per project with uv



Today you will add

- Confident command of Python syntax, types and collections
- Control flow and functions for reusable, modular code
- Robust error handling and file I/O for real datasets
- Working familiarity with the core AI library stack
- A finished end-to-end mini ML workflow

Workshop roadmap



Four sections, each ending in a short hands-on exercise

Workshop setup: libraries and data

```
# uv is already installed from Workshop 1
uv init workshop2 && cd workshop2
uv python pin 3.13

uv add numpy pandas matplotlib scikit-learn
uv add joblib jupyterlab
uv add torch                # large download, CPU wheel
uv add tensorflow            # large download, optional

uv run python -c "import numpy, pandas, sklearn"
uv run jupyter lab          # or open the folder in VS Code

data/sensors.csv           # place the dataset here
```

- Dataset: sensors.csv — download link in the workshop invitation
- Save it as data/sensors.csv, the path used in Sections 3 and 4
- Columns: ts, node, temp_c, t_max, t_min — one row per reading
- torch and tensorflow are large downloads — run these before the session
- uv.lock pins exact versions; commit it so everyone runs the same stack

Syntax and primitives

- Indentation defines blocks — no braces, no semicolons
- Names are references to objects; variables are untyped, values are not
- Primitives: int, float, complex, bool, str, NoneType
- Dynamic typing with optional type hints for clarity and tooling
- f-strings are the standard way to format output

```
temperature = 36.6           # float
sensor_id   = "NODE-07"     # str
readings    = 128           # int
calibrated  = True          # bool

def to_f(c: float) -> float:
    return c * 9 / 5 + 32

print(f"{sensor_id}:
{to_f(temperature):.1f}F")
# NODE-07: 97.9F
```

Built-in types at a glance

Type	Example	Mutable	Typical use in AI work
int / float	42, 3.14	No	Counters, learning rates, metrics
str	"label"	No	Class names, file paths, feature keys
bool	True / False	No	Masks, flags, filter conditions
list	[1, 2, 3]	Yes	Ordered batches, growing collections
tuple	(224, 224, 3)	No	Fixed shapes, coordinates, safe returns
dict	{"lr": 1e-3}	Yes	Hyperparameters, configs, JSON records
set	{"cat", "dog"}	Yes	Unique labels, fast membership tests

Sequential collections

```
labels = ["cat", "dog", "bird", "fish"]
```

```
labels[0]      # 'cat'    first element
```

```
labels[-1]     # 'fish'   last element
```

```
labels[1:3]    # ['dog', 'bird'] slice
```

```
labels.append("cow")  # grows in place
```

```
upper = [w.upper() for w in labels if len(w) == 3]  
# ['CAT', 'DOG', 'COW']
```

```
shape = (224, 224, 3)  # tuple: fixed, safe
```

```
h, w, c = shape        # unpacking
```

List vs tuple

Reach for a list when the collection grows or changes. Reach for a tuple when the values are fixed — image shapes, coordinates, multiple return values.

Comprehensions

A comprehension replaces a build-a-list loop with one readable line. It is the idiomatic way to transform and filter data before it reaches a library.

Dictionaries and sets

```
reading = {"node": "node-a", "temp_c": 23.0}

reading["temp_c"]           # 23.0, KeyError if absent
reading.get("humidity", 0)  # 0, a safe default
reading["node_id"] = 7      # add or overwrite

for key, value in reading.items():
    print(key, value)

"temp_c" in reading         # True, tests the keys

nodes = {"node-a", "node-b", "node-a"}
len(nodes)                  # 2, duplicates dropped
```

- A dict maps keys to values; keys must be immutable, values need not be
- `.get()` returns a default instead of raising `KeyError`
- `.items()` yields key/value pairs — the normal way to loop a dict
- A set holds unique items and tests membership in constant time
- Dicts return later as `csv.DictReader` rows, JSON records and hyperparameters

Exercise 1: readings and collections

Your task

```
# sensor_readings.py
readings = [21.5, 23.0, 19.8, 25.1, 22.4]
labels    = ("node-a", "node-b", "node-c")

# 1. Print the type of each variable
# 2. Report the first, last and middle reading
# 3. Build a list of readings above 22.0
# 4. Format a one-line summary with an f-string
#     e.g. "node-a: 5 readings, mean 22.36"
# 5. Why can labels not be appended to?
```

Hints

```
# type(x) tells you what you are holding
# negative indices count from the end
# len(seq) // 2 lands on the middle item
# [expr for item in seq if condition]
# f"{value:.1f}" rounds inside the string
# a tuple cannot be changed once created
```

Work in pairs, 10 minutes. Check each step in the REPL as you go.

Try each hint on its own before writing the full script.

Solution 1: readings and collections

Solution

```
print(type(readings), type(labels))

first, last = readings[0], readings[-1]
middle = readings[len(readings) // 2]

hot = [r for r in readings if r > 22.0]

mean = sum(readings) / len(readings)
print(f"{labels[0]}: {len(readings)} readings, "
      f"mean {mean:.1f}")
# node-a: 5 readings, mean 22.36

# labels is a tuple - immutable by design
```

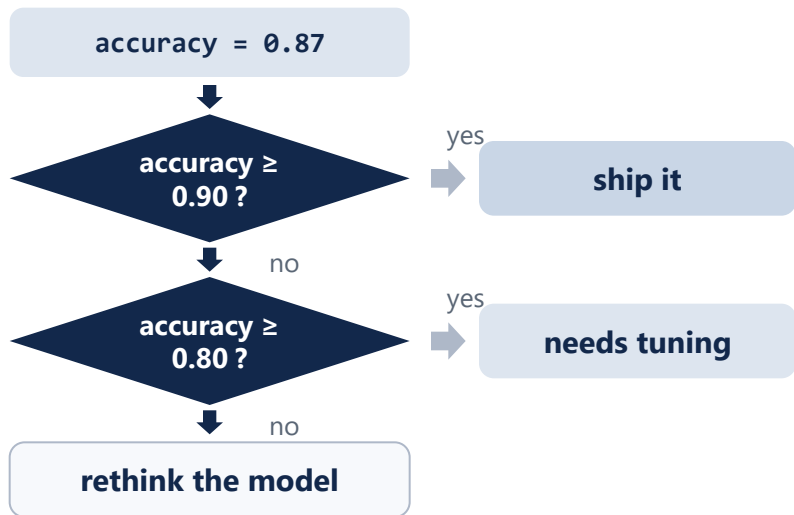
Why it works

```
# type() confirms list vs tuple
# readings[-1] beats len(readings) - 1
# the comprehension filters in one pass
# mean is derived, never stored twice
# the f-string keeps formatting next to
#   the data it describes
# labels stays a tuple, so the node names
#   cannot be changed by accident
```

Run it end to end, then change one reading and predict the output.

Common slip: reaching for readings[6] and hitting IndexError.

Conditional branching



Operators: == != < > and or not
Falsy: empty containers, 0 and None
match / case for multi-way dispatch (3.10+)

```
accuracy = 0.87
```

```
if accuracy >= 0.90:  
    verdict = "ship it"  
elif accuracy >= 0.80:  
    verdict = "needs tuning"  
else:  
    verdict = "rethink the model"
```

```
# one-line conditional expression  
status = "pass" if accuracy > 0.5 else "fail"
```

```
if not predictions:      # empty list is falsy  
    raise ValueError("no predictions returned")
```

Iteration constructs

```
for name in labels:           # iterate values
    print(name)

for i, name in enumerate(labels): # index + value
    print(i, name)

for name, score in zip(labels, scores): # in parallel
    print(f"{name}: {score:.2f}")

epoch = 0
while epoch < 10 and loss > 0.01: # condition-driven
    loss = train_one_epoch()
    epoch += 1

for row in rows:
    if row.is_empty: continue # skip
    if row.id == target: break # stop early
```

Prefer direct iteration

Loop over the collection itself. Use `enumerate` when you also need the position, and `zip` when two sequences advance together.

Loop with intent

`break` exits the loop, `continue` skips one pass, and `else` after a loop runs only when no `break` occurred — useful in search routines.

Function definitions and modules

```
def normalize(values, lo=0.0, hi=1.0):
    """Scale values into the [lo, hi] range."""
    lowest, highest = min(values), max(values)
    span = highest - lowest or 1.0
    return [lo + (v - lowest) / span * (hi - lo)
            for v in values]

normalize(readings)           # positional
normalize(readings, hi=255)    # keyword

def summarize(*values, **options): # flexible args
    ...

# preprocessing.py -> reusable module
from preprocessing import normalize
```

- Default arguments make functions easy to call and hard to misuse
- Docstrings document intent; your IDE and teammates both read them
- Return values, not printed output — printing is a debugging habit
- Keep functions small and single-purpose so they can be tested
- Group related functions into modules and import them across notebooks

Exercise 2: branching, loops and functions

Your task

```
scores = [0.91, 0.47, 0.78, 0.55, 0.96]

# 1. Write grade(score) returning "high",
#     "mid" or "low" using if / elif / else
# 2. Give it threshold arguments with defaults
# 3. Loop with enumerate and print index,
#     score and grade for every entry
# 4. Stop early once a score above 0.95 is
#     found, and report which index it was
# 5. Save it as grading.py and import it
```

Hints

```
# if / elif / else stops at the first match
# def f(x, high=0.9) sets a default value
# enumerate(seq) yields index and value
# break leaves the loop immediately
# return a value - do not print inside
# save it in grading.py so you can import it
```

15 minutes. Keep the function free of print statements.

Try each hint on its own before writing the full script.

Solution 2: branching, loops and functions

Solution

```
def grade(score, high=0.90, mid=0.60):  
    """Bucket a score into three bands."""  
    if score >= high:  
        return "high"  
    elif score >= mid:  
        return "mid"  
    return "low"  
  
for i, s in enumerate(scores):  
    print(i, s, grade(s))  
    if s > 0.95:  
        print(f"early stop at index {i}")  
        break
```

Import grade() from another file to prove it is reusable.

Why it works

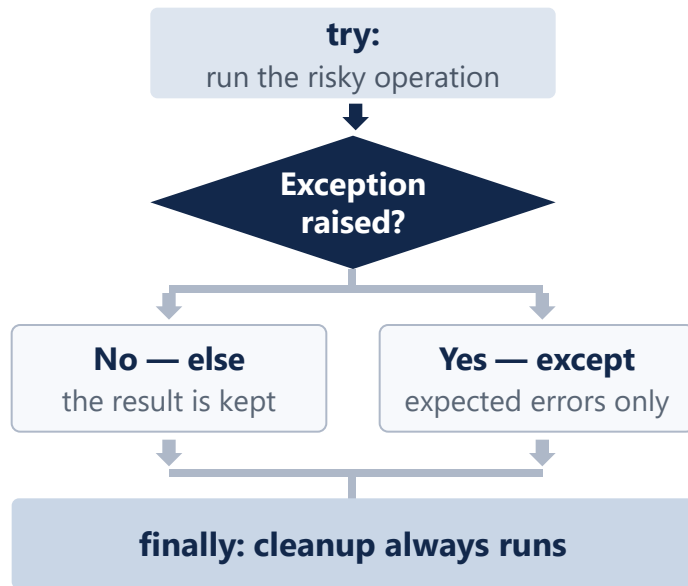
```
# defaults let callers ignore thresholds  
# early returns keep the branches flat  
# enumerate avoids range(len(scores))  
# break stops the scan as soon as the  
#   goal is met - no wasted iterations  
# the function returns instead of printing,  
#   so it can be imported and tested
```

Common slip: printing inside the function instead of returning.

Exception management

```
try:
    value = float(raw_input)
except ValueError:
    value = float("nan")          # handle the known case
except (TypeError, KeyError) as err:
    logging.warning("bad record: %s", err)
    raise                        # re-raise if can't fix
else:
    records.append(value)         # ran without error
finally:
    cleanup()                    # always runs

if lr <= 0:
    raise ValueError("learning rate must be positive")
```



A bare except hides real bugs — name what you can recover from, re-raise the rest.

Reading a traceback

Traceback (most recent call last):

File "clean.py", line 42, in <module>

main()

File "clean.py", line 31, in main

value = convert(row["temp_c"])

File "clean.py", line 18, in convert

return float(raw) * 9 / 5 + 32

^^^^^^^^^^

ValueError: could not convert string to float: 'n/a'

1. read the last line first

2. then find the deepest frame in a file you wrote

3. fix it there, not where the traceback starts

↑
read bottom-up

<module> · clean.py line 42

main()

main · clean.py line 31

value = convert(row["temp_c"])

convert · clean.py line 18

return float(raw) * 9 / 5 + 32

ValueError

could not convert string to float: 'n/a'

Paste this final line for help, never a screenshot

File input and output

```
from pathlib import Path
import csv, json

path = Path("data") / "sensors.csv"
with open(path, "r", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))    # auto-closed

with open("clean.json", "w", encoding="utf-8") as f:
    json.dump(rows, f, indent=2)

with open(path, encoding="utf-8") as f:    # streaming
    for line in f:
        process(line.strip())
```

- Always use `with` — the file closes even if an error is raised
- Modes: `r` read, `w` overwrite, `a` append, `rb/wb` for binary
- Set `encoding="utf-8"` explicitly to avoid platform surprises
- Use `pathlib` for paths that work on WSL2, Linux and Windows alike
- Stream large files line by line instead of loading them whole

File input and output

```
from pathlib import Path
import csv, json

path = Path("data") / "sensors.csv"
with open(path, "r", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))    # auto-closed

with open("clean.json", "w", encoding="utf-8") as f:
    json.dump(rows, f, indent=2)

with open(path, encoding="utf-8") as f:    # streaming
    for line in f:
        process(line.strip())
```

- Always use `with` — the file closes even if an error is raised
- Modes: `r` read, `w` overwrite, `a` append, `rb/wb` for binary
- Set `encoding="utf-8"` explicitly to avoid platform surprises
- Use `pathlib` for paths that work on WSL2, Linux and Windows alike
- Stream large files line by line instead of loading them whole

Exercise 3: robust file processing

Your task

```
# readings.csv contains blanks and bad rows

# 1. Open it inside a with block, utf-8
# 2. Convert each temp_c value to float
# 3. Catch ValueError and count bad rows
#    instead of letting the script crash
# 4. Write the clean values to clean.json
# 5. Raise a clear error if the file is
#    missing - do not fail silently
# 6. Print how many rows you had to skip
```

20 minutes. Corrupt a row on purpose and confirm the script survives.

Hints

```
# with open(...) closes the file for you
# csv.DictReader gives you dict-like rows
# float("") raises ValueError
# wrap only the risky line in try
# Path.exists() checks before you read
# json.dump writes the list back out
```

Try each hint on its own before writing the full script.

Solution 3: robust file processing

Solution

```
import csv, json
from pathlib import Path

path = Path("readings.csv")
if not path.exists():
    raise FileNotFoundError(f"missing: {path}")

clean, bad = [], 0
with open(path, encoding="utf-8") as f:
    for row in csv.DictReader(f):
        try:
            clean.append(float(row["temp_c"]))
        except ValueError:
            bad += 1
```

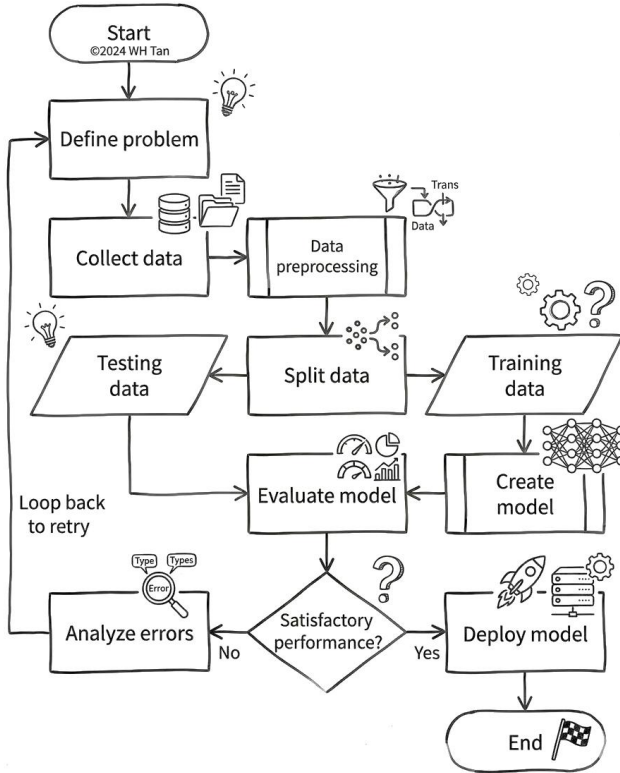
Add a print of the skipped count to close the loop.

Why it works

```
# the guard fails fast with a clear message
# try wraps one line only, so real bugs
#   still surface instead of being hidden
# bad rows are counted, not silently lost
# with closes the file even on error
# an explicit encoding makes the script
#   behave the same on WSL2 and Windows
```

Common slip: a bare except that hides typos as if they were bad data.

Machine learning workflow



What is JupyterLab?

Runs in your browser

One workspace for notebooks, files, terminals and plots — no install per project.

Code in cells

Run a few lines at a time and see the result immediately underneath.

Backed by a kernel

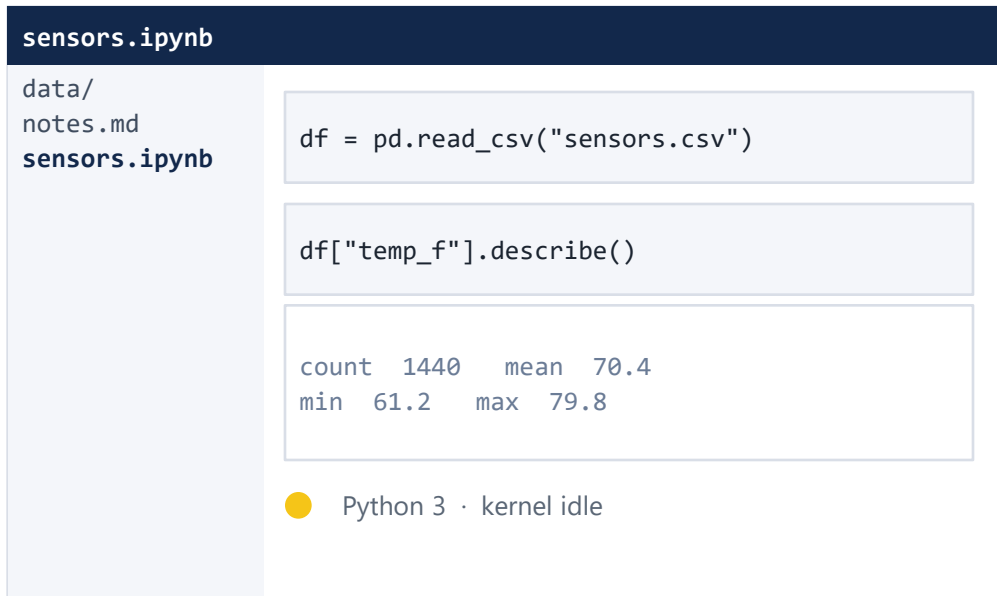
A live Python process keeps your variables in memory between cells.

Why it earns a place next to your editor

Exploring data, checking an assumption, or explaining a result to someone — JupyterLab keeps the code, the output and the narrative in one document.

Same Python you already write — a different interface for working with it.

A tour of the JupyterLab window



The screenshot shows the JupyterLab interface. On the left is a file browser with a dark blue header labeled 'sensors.ipynb'. Below the header, it lists 'data/' and 'notes.md'. The main area contains three code cells. The first cell has the code `df = pd.read_csv("sensors.csv")`. The second cell has the code `df["temp_f"].describe()`. The third cell shows the output of the second cell as a text table:

count	1440	mean	70.4
min	61.2	max	79.8

. At the bottom of the main area, there is a yellow dot icon followed by the text 'Python 3 · kernel idle'.

1

File browser

Notebooks, data and notes open side by side in one workspace.

2

Cells

Write a few lines, run with Shift+Enter, keep going.

3

Inline output

Tables and figures appear directly beneath the cell.

4

Kernel

The live Python session holding your variables — restart to reset state.

Everything on this slide runs in a browser tab — nothing to install per notebook.

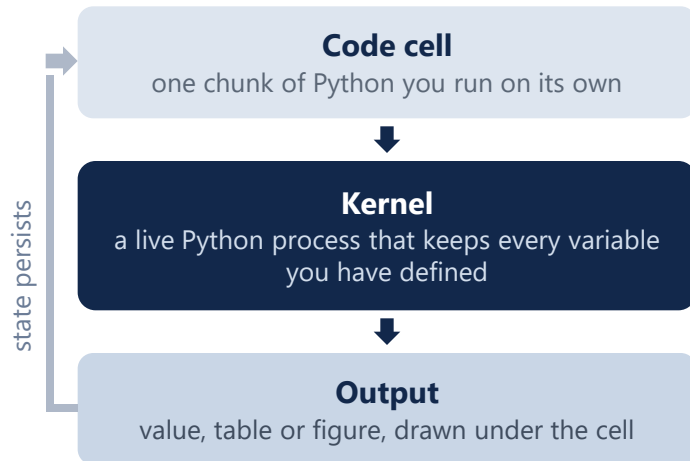
Notebooks in your workflow

```
# cell 1
import pandas as pd
df = pd.read_csv("data/sensors.csv")

# cell 2 - reuses everything cell 1 defined
df["temp_f"] = df["temp_c"] * 9/5 + 32
df.head(2)

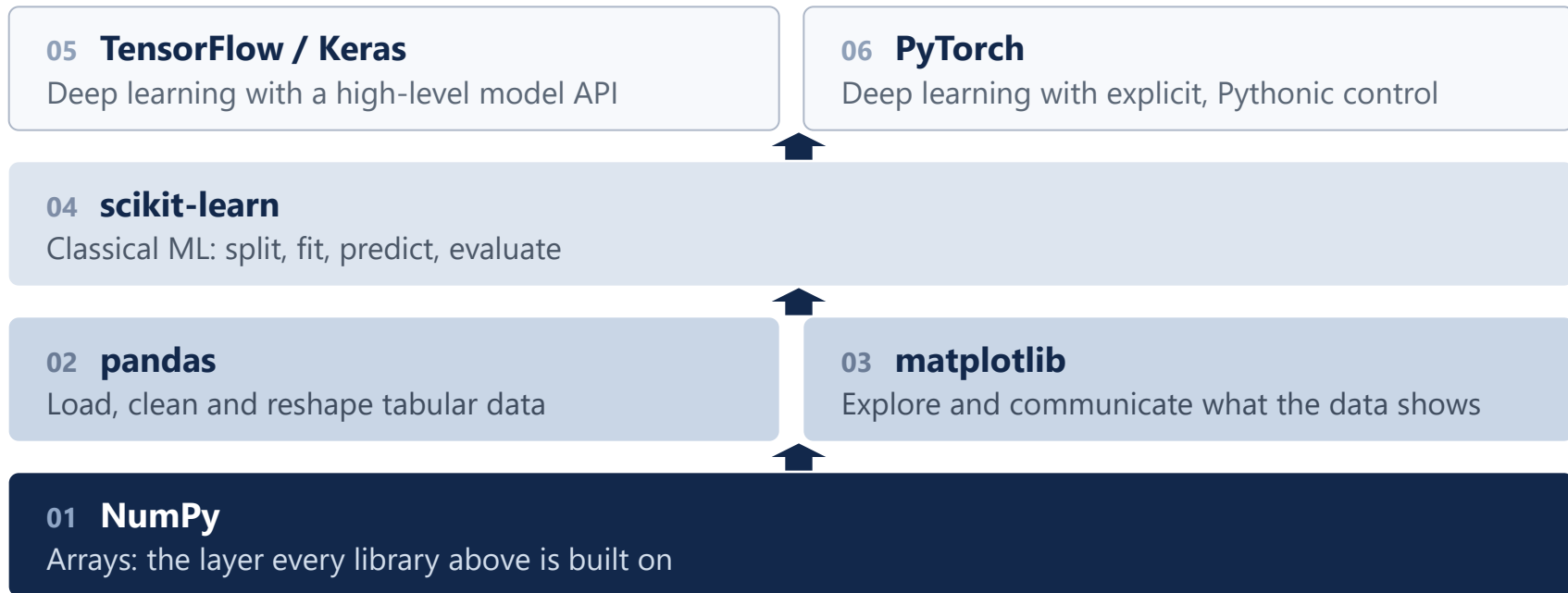
#      node  temp_c  temp_f
# 0    a01    21.4    70.5
# 1    a02    19.8    67.6

# cell 3 - figures render under the cell
df["temp_f"].plot(kind="hist", bins=20)
```



Notebook to explore and explain. Module to ship.

The core AI library stack



Each library assumes the Python fundamentals from Sections 1-3 — the stack is only as solid as the code around it.

Arrays with NumPy

```
import numpy as np

a = np.array([[21.5, 23.0], [19.8, 25.1]])
a.shape, a.dtype      # (2, 2) float64

a.mean(), a.mean(axis=0)  # all, then per column
a * 9 / 5 + 32           # vectorised, no loop

a[:, 0]                 # first column
a[a > 22]               # boolean mask

np.asarray(df[["temp_c"]])  # pandas -> ndarray
torch.from_numpy(a).float() # ndarray -> tensor
```

- pandas, scikit-learn and both DL frameworks all sit on the ndarray
- One dtype per array — mixed types fall back to slow object arrays
- Vectorised expressions replace Python loops and run in compiled C
- Slicing returns a view, not a copy — mutating it changes the original
- Broadcasting stretches the smaller shape to match the larger one

Tabular data with pandas

```
import pandas as pd

df = pd.read_csv("sensors.csv")
df.head()          # first rows
df.info()          # dtypes and null counts
df.describe()      # summary statistics

df["temp_f"] = df["temp_c"] * 9 / 5 + 32  # new column
hot = df[df["temp_c"] > 30]               # boolean filter

df.groupby("node")["temp_c"].mean()      # split-apply-combine
df = df.dropna(subset=["temp_c"])        # handle missing values

X = df[["temp_c", "humidity"]]           # features
y = df["fault"]                          # target
```

Series and DataFrame

A Series is one labelled column; a DataFrame is a table of them. Both carry an index, which is what makes joins and alignment automatic.

Where models begin

Almost every scikit-learn workflow starts with a clean DataFrame split into a feature matrix X and a target vector y .

Cleaning and reshaping data

```
df.isna().sum()          # where are the gaps?
df = df.dropna(subset=["temp_c"])
df["hum"] = df["hum"].fillna(df["hum"].median())

df = df.drop_duplicates()
df["node"] = df["node"].str.strip().str.lower()
df["ts"]   = pd.to_datetime(df["ts"])

daily = (df.groupby(df["ts"].dt.date)["temp_c"]
         .agg(["mean", "max"]))
wide  = df.pivot_table(index="ts",
                       columns="node", values="temp_c")
merged = df.merge(nodes, on="node", how="left")
```

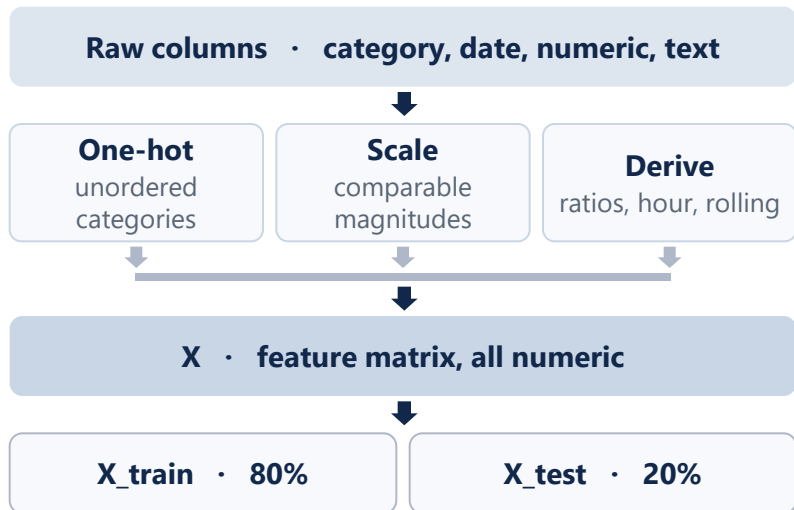
Missing data is a decision

Dropping rows is honest but costly; filling with a median keeps the row and invents a value. Record which you chose — it changes the model.

Tidy before you model

One row per observation, one column per variable. `groupby`, `pivot_table` and `merge` get you there far faster than writing loops.

From columns to features



Fit every transform on the training split only, then apply it to the test data.

```
from sklearn.preprocessing import
StandardScaler
from sklearn.model_selection import (
    train_test_split)

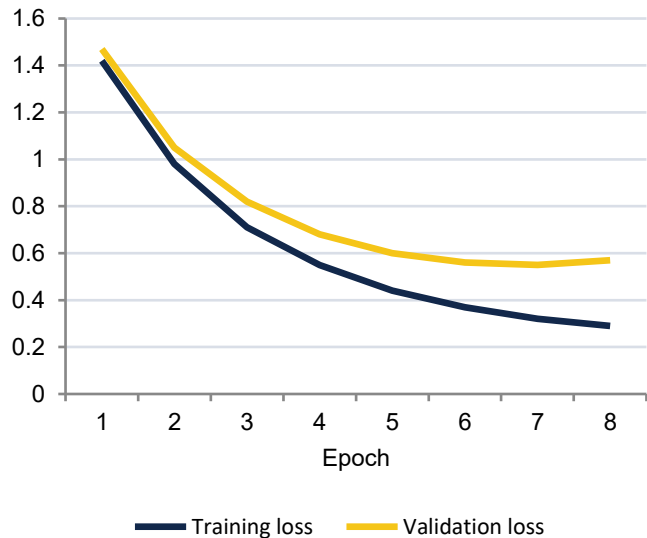
df["temp_range"] = df["t_max"] - df["t_min"]
df["hour"] = df["ts"].dt.hour
df["rolling"] = df["temp_c"].rolling(3).mean()
X = pd.get_dummies(df[["node", "hour"]],
                    drop_first=True)
X_train, X_test = train_test_split(
    X, test_size=0.2, random_state=42)
# fit on the training split only
scaler = StandardScaler().fit(X_train)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
```

Data visualization with matplotlib

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(history.history["loss"], label="train")
ax.plot(history.history["val_loss"], label="val")
ax.set_xlabel("Epoch")
ax.set_ylabel("Loss")
ax.set_title("Training curve")
ax.legend()
fig.savefig("loss.png", dpi=150, bbox_inches="tight")
```

- Use the figure/axes API — it scales to multi-panel figures
- Histograms and scatter plots expose outliers a summary table hides
- Always label axes and units; an unlabelled plot proves nothing
- Save figures to file for reports rather than screenshotting



Reading the curve: the gap between the two lines is where overfitting begins.

Machine learning with scikit-learn

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import make_pipeline
from sklearn.metrics import accuracy_score, classification_report

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

model = make_pipeline(StandardScaler(),
                      RandomForestClassifier(n_estimators=200))
model.fit(X_train, y_train)          # learn
pred = model.predict(X_test)         # apply
print(accuracy_score(y_test, pred)) # evaluate
```

Split

Hold out test data before you touch it

Fit

Learn parameters from the training set only

Predict

Apply the fitted model to unseen data

Evaluate

Accuracy, precision, recall, confusion matrix

Evaluating beyond accuracy

	Predicted +	Predicted -
Actual +	TP true positives	FN missed cases
Actual -	FP false alarms	TN true negatives

Recall = $TP / (TP + FN)$ · across the actual-positive row

Precision = $TP / (TP + FP)$ · down the predicted-positive column

Accuracy

Fine when classes are balanced, misleading the moment they are not

Precision & recall

Precision guards against false alarms, recall against missed cases

F1 and ROC-AUC

F1 balances the two; ROC-AUC scores ranking across all thresholds

Confusion matrix

Shows which classes get mistaken for which — always look at it

Regression metrics

MAE in the units you care about; RMSE when large errors hurt more

Choose the metric before you train, based on what a false positive and a false negative actually cost.

Section 4 · Choose a metric that matches the cost of being wrong

Exercise 4: an end-to-end mini workflow

Your task

```
# Use the built-in digits dataset

# 1. Load it into a pandas DataFrame
# 2. Print the shape and the class counts
# 3. Show one digit with plt.imshow()
# 4. Split 80/20, stratified, fixed seed
# 5. Fit a RandomForestClassifier and report
#     accuracy on the held-out test set
# 6. Bonus: print a confusion matrix and
#     see which digits get mixed up
```

Hints

```
# load_digits(as_frame=True) returns pandas
# 64 columns = one 8x8 grayscale image
# value_counts() checks class balance
# imshow(d.images[0], cmap="gray") plots it
# stratify keeps all ten digits balanced
# model.score() returns accuracy directly
```

25 minutes. This is a rehearsal for the full lab at the end.

Try each hint on its own before writing the full script.

Solution 4: an end-to-end mini workflow

Solution

```
import pandas as pd, matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

d = load_digits(as_frame=True)
df = d.frame
print(df.shape, df["target"].value_counts())
plt.imshow(d.images[0], cmap="gray"); plt.show()
Xtr, Xte, ytr, yte = train_test_split(
    d.data, d.target, test_size=0.2,
    stratify=d.target, random_state=42)
m = RandomForestClassifier().fit(Xtr, ytr)
print(m.score(Xte, yte))      # around 0.97
```

Swap in a different classifier and compare the score.

Why it works

```
# explore before you model - shape and
#   class counts catch bad data early
# each row is a flattened 8x8 image
# stratify keeps the ten classes even
# the test set is held out before fitting
# score() is accuracy; a confusion matrix
#   shows which digits get mixed up
# the same four beats every time:
#   split, fit, predict, evaluate
```

Common slip: fitting on all the data, then reporting test accuracy.

Deep learning: Keras and PyTorch

TensorFlow / Keras

```
import keras

model = keras.Sequential([
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dense(3, activation="softmax")])

model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

model.fit(X_train, y_train, epochs=10,
          validation_split=0.2)
model.evaluate(X_test, y_test)
```

Declarative and concise — ideal for standard architectures.

PyTorch

```
import torch
from torch import nn

model = nn.Sequential(
    nn.Linear(n_features, 64), nn.ReLU(),
    nn.Linear(64, 3))

loss_fn = nn.CrossEntropyLoss()
opt = torch.optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(10):          # explicit loop
    opt.zero_grad()
    loss = loss_fn(model(X_train), y_train)
    loss.backward(); opt.step()
```

Explicit training loop — full control, easier to debug and extend.

Anatomy of an artificial neuron

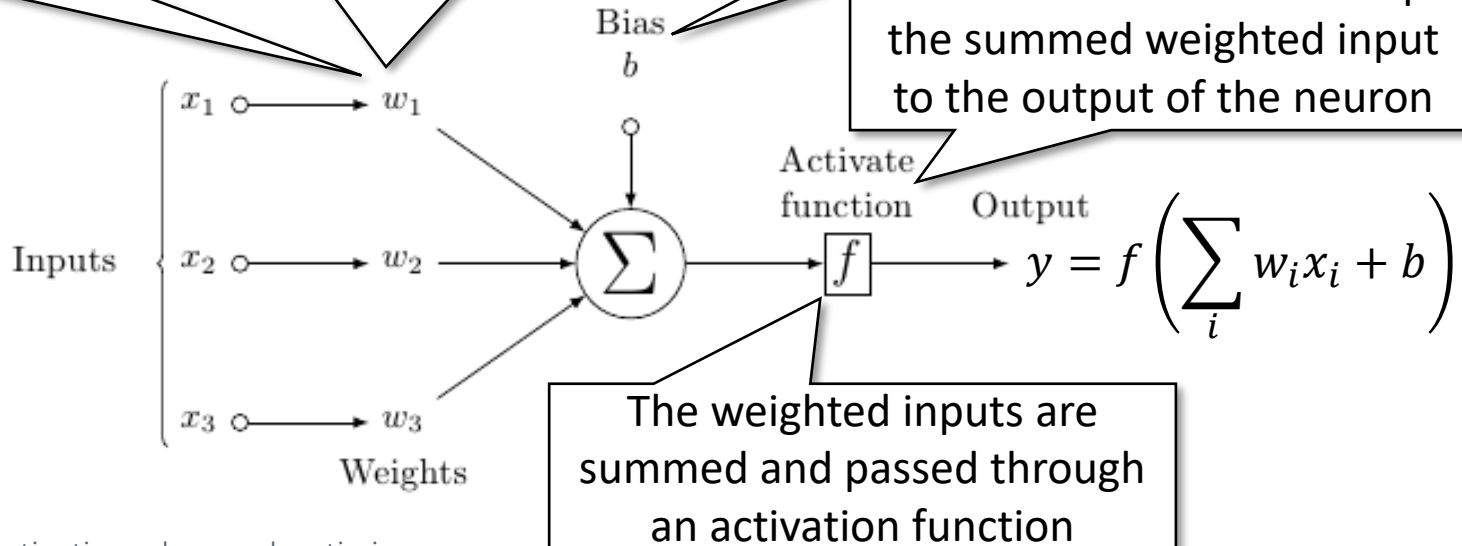
The weights are expressions of importance of respective inputs to the output.

The weights are initialized to small random values (between 0 to 0.3)

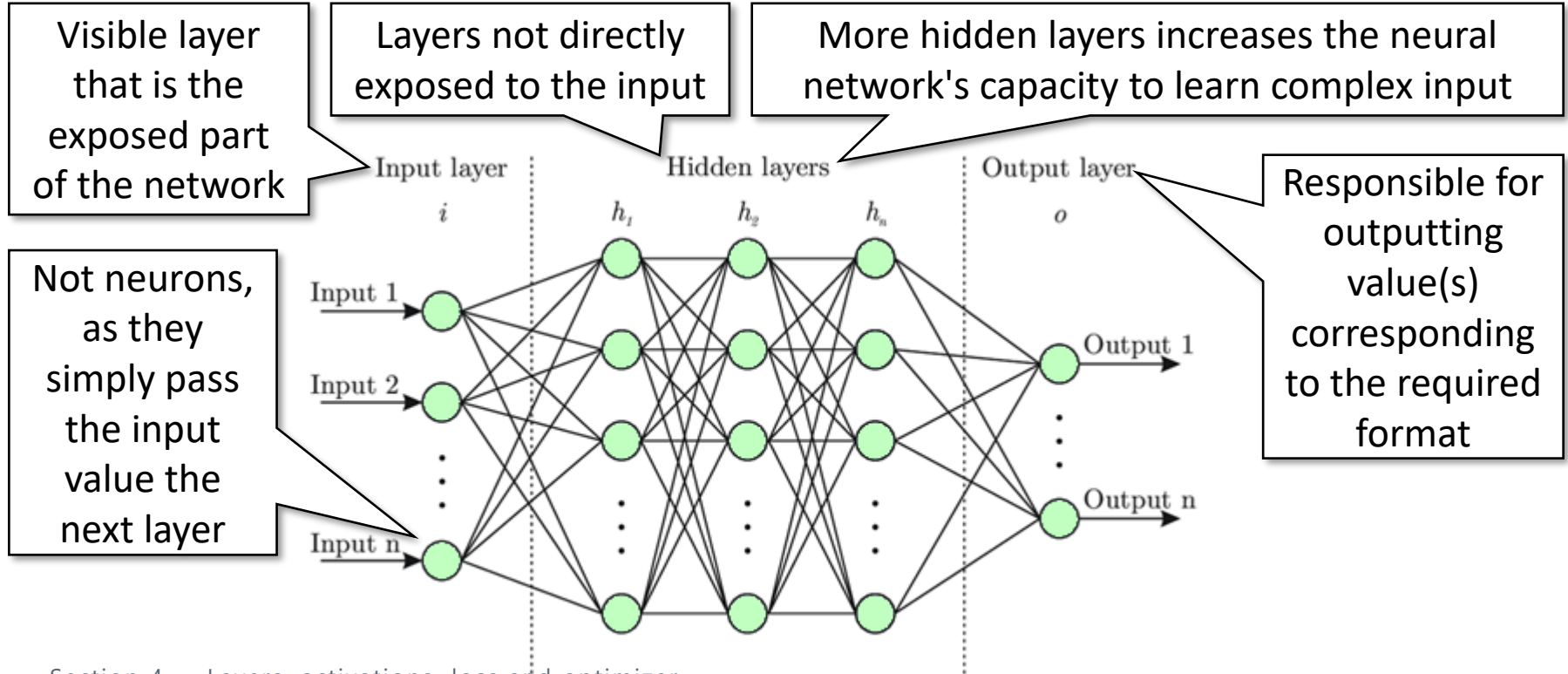
The bias is an input with value of 1.0 and must also be weighted.

The activation function maps the summed weighted input to the output of the neuron

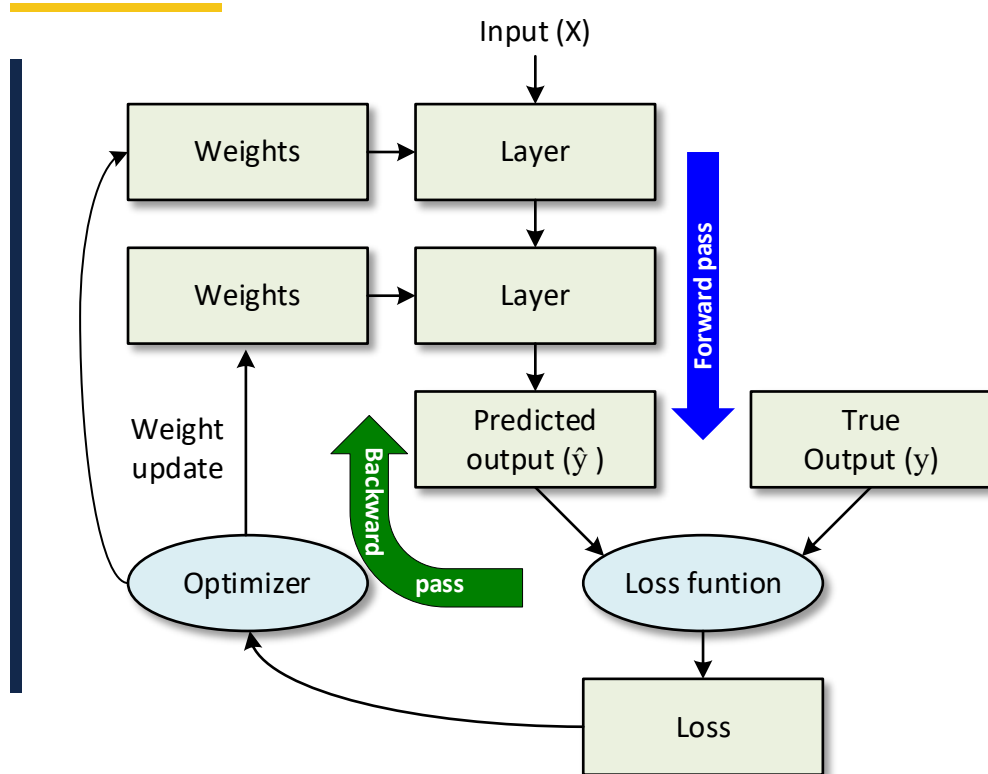
A neuron with three inputs requires four weights: one for each input and one for the bias.



Anatomy of a neural network



Inside the training loop



Forward

Data flows through the layers to a prediction

Measure

The loss scores how wrong that prediction is

Backward

Optimizer computes the gradient of the loss

Update

The optimizer nudges every weight one small step

Repeats every batch, then every epoch

Choosing your framework

01	First model	Keras reaches a working baseline fastest with very little code
02	Custom research	PyTorch dominates papers and reference implementations
03	Deployment	TF Serving and LiteRT (formerly TF Lite); PyTorch offers TorchScript and ExecuTorch
04	Debugging	Both run eagerly — step through a PyTorch loop with a normal debugger
05	What transfers	Layers, loss, optimizer and the training loop are identical in both

Learn one framework properly and the second one takes days, not weeks — the concepts are shared.

Section 4 · Both are production-grade — choose for the task, not the tribe

CPU vs GPU: two different machines

CPU — a few fast, general cores

- 8–64 cores built for branching logic
- Large caches, high clock speeds
- Runs the OS, I/O and pandas well
- Good for tabular data, small models
- Matrix maths runs a chunk at a time
- Mid-size training takes hours or days



GPU — thousands of small cores

- Thousands of cores plus tensor cores
- High-bandwidth memory feeds them all
- Deep learning is dense matrix maths
- Often 10–50× faster to train
- VRAM caps model and batch size
- Generative AI needs one, in practice

Rule of thumb: classical ML and data prep on the CPU; anything with layers and epochs on the GPU.

Why the GPU changed deep learning

01 Throughput

One modern GPU trains what once needed a rack of CPU servers

02 Batch size

More VRAM allows larger batches and steadier gradient updates

03 Model scale

Transformers and diffusion models need GPU memory

04 Iteration speed

Faster epochs mean more experiments per day

05 Inference cost

Batched GPU serving cuts the cost per token, image or prediction

CPUs still do the data loading, feature work and classical ML — the GPU is for the tensor maths.

Installing GPU-enabled TensorFlow

01	Check the driver	Run <code>nvidia-smi</code> — it must report your GPU and a recent driver version
02	Use Linux or WSL2	Native Windows GPU support stopped at TF 2.10; install inside WSL2 instead
03	Create the env	<code>uv venv --python 3.13</code>
04	Install	<code>uv pip install "tensorflow[and-cuda]"</code> — the extra pulls matching CUDA and cuDNN wheels
05	Verify	<code>python -c "import tensorflow as tf; print(tf.config.list_physical_devices('GPU'))"</code>

An empty device list means the driver, the WSL2 setup or the [and-cuda] extra is missing — not your code.

Installing GPU-enabled PyTorch

01	Check the driver	<code>nvidia-smi</code> shows the GPU and the highest CUDA version the driver supports
02	Let uv choose	<code>uv pip install torch torchvision --torch-backend=auto</code> detects the driver for you
03	Or pin it	<code>uv pip install torch torchvision --index-url https://download.pytorch.org/whl/cu124</code>
04	Verify	<code>python -c "import torch; print(torch.cuda.is_available(), torch.cuda.get_device_name(0))"</code>
05	Use the device	<code>device = 'cuda' if torch.cuda.is_available() else 'cpu' then model.to(device)</code>

No NVIDIA card? Install the default wheel — Apple Silicon uses `device='mps'`, everything else falls back to CPU.

Section 4 · Pick the wheel that matches your driver, then confirm it

Saving and loading trained models

```
import joblib
from pathlib import Path

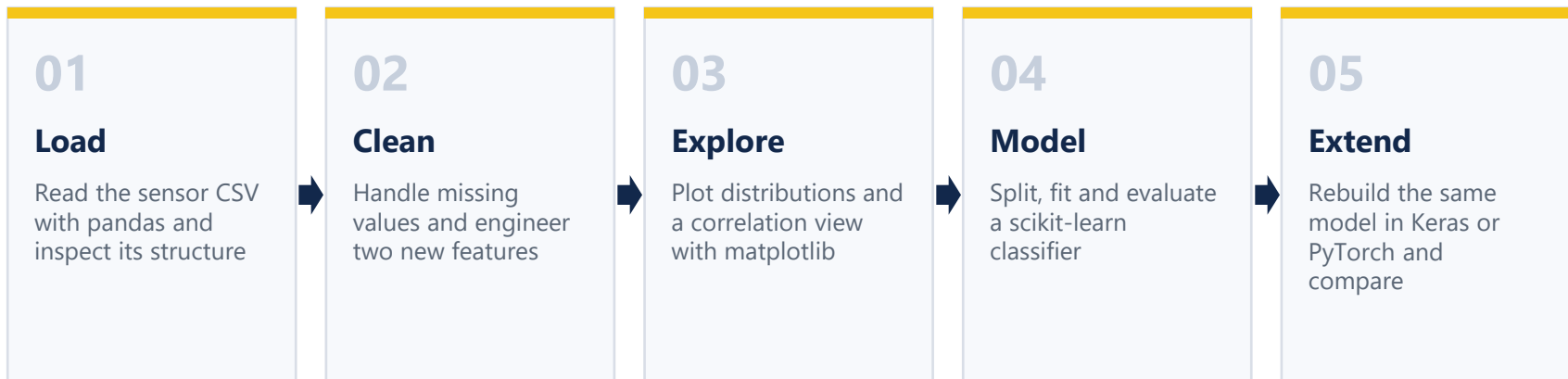
Path("models").mkdir(exist_ok=True)
joblib.dump(pipe, "models/rf_v1.joblib") # sklearn
loaded = joblib.load("models/rf_v1.joblib")
loaded.predict(X_test[:5])

model.save("models/keras_v1.keras") # Keras 3
keras.saving.load_model("models/keras_v1.keras")

torch.save(net.state_dict(), "models/net_v1.pt")
net.load_state_dict(torch.load("models/net_v1.pt"))
```

- Save the fitted pipeline, not just the estimator — scalers must travel with it
- Keras 3 writes the .keras format; .h5 is legacy
- PyTorch saves a `state_dict` — load it into the same class definition
- joblib artifacts are version-sensitive: pin library versions with the file
- Never load a model artifact you did not produce yourself

Hands-on lab and next steps



Keep going after today

- Version your notebooks and lab code with git as you work
- Pin dependencies with uv so your results stay reproducible
- Rebuild the lab on a dataset from your own discipline
- Read the official docs — they are the fastest path to fluency

Bring Sections 1-4 together in one end-to-end workflow